Especificações de Ambiente e Infraestrutura AutoShopping

Este documento apresenta os requisitos necessários para executar, testar e realizar manutenção no sistema AutoShopping.

As versões abaixo foram verificadas no ambiente de desenvolvimento.

1. Ambiente utilizado no desenvolvimento

Componente	**Versão utilizada**
Sistema operacional	Windows 10 Pro 22H2
Build do Windows	19045
Arquitetura	x64 — 64 bits
Memória RAM instalada	15,9 GB
Node.js	22.18.0
npm	10.9.3
MySQL Server	8.0.46 Community
MySQL Workbench	8.0.47
Google Chrome	141.0.7390.108
Visual Studio Code	1.141.0 x64
REST Client	0.25.1
Live Server	5.7.10
Git	2.52.0 para Windows

---

2. Requisitos de hardware

Configuração mínima

- Processador Intel ou AMD de 64 bits.
- Processador com pelo menos 2 núcleos.
- 4 GB de memória RAM.
- 2 GB de espaço livre no armazenamento.
- Conexão de rede local.
- Conexão com a internet para instalação das dependências, consulta de CEP e carregamento dos gráficos.

Configuração recomendada

- Processador Intel Core i3, AMD Ryzen 3 ou superior.
- Processador com 4 núcleos ou mais.

- 8 GB de memória RAM ou mais.
- 5 GB de espaço livre.
- Armazenamento SSD.
- Arquitetura x64.
- Conexão estável com a internet.

O computador utilizado no desenvolvimento possui aproximadamente **16 GB de RAM**, sendo mais que suficiente para executar o frontend, backend, banco de dados, Workbench e VS Code simultaneamente.

---

3. Sistema operacional

Sistema testado

- Windows 10 Pro.
- Versão 22H2.
- Build 19045.
- Arquitetura de 64 bits.

Sistemas recomendados

- Windows 10 x64.
- Windows 11 x64.

O backend também pode funcionar em Linux ou macOS, desde que Node.js e MySQL estejam instalados, mas o ambiente oficialmente testado neste projeto foi o Windows 10 x64.

---

4. Arquitetura do computador

A arquitetura recomendada é:

- **x64 ou 64 bits**

Devem ser instaladas as versões de 64 bits do:

- Node.js;
- MySQL Server;
- MySQL Workbench;
- Visual Studio Code;
- Google Chrome.

A arquitetura x86 de 32 bits não é recomendada por limitações de compatibilidade, memória e suporte das ferramentas atuais.

---

5. Node.js e npm

Versão utilizada

- Node.js: `22.18.0`
- npm: `10.9.3`

Versão mínima

As bibliotecas Express e bcrypt utilizadas exigem:

- Node.js `18` ou superior.

Entretanto, para evitar diferenças no funcionamento, recomenda-se utilizar:

- Node.js `22.x` de 64 bits;
- npm `10.x` ou superior.

O npm é instalado automaticamente junto com o Node.js.

Para verificar as versões:

```
node --version
npm.cmd --version
```

Caso o PowerShell bloqueie o comando `npm`, utilize `npm.cmd`.

---

6. Banco de dados

Banco utilizado

- MySQL Community Server 8.0.46.
- Porta padrão: `3306`.
- Banco do projeto: `db_ecom`.
- Codificação: `utf8mb4`.
- Collation: `utf8mb4_unicode_ci`.

Versão recomendada

- MySQL Server 8.0 ou superior.

Tabelas do sistema

O banco possui sete tabelas principais:

1. `usuarios`
2. `categorias`
3. `produtos`
4. `estoques`
5. `pedidos`
6. `itens_pedidos`
7. `entregas`

O usuário configurado no MySQL deve possuir permissão para:

- Criar banco de dados;
- Criar tabelas;
- Consultar registros;
- Inserir registros;
- Atualizar registros;
- Apagar registros.

---

7. MySQL Workbench

Versão utilizada

- MySQL Workbench 8.0.47.

Finalidade

O Workbench é utilizado para:

- Visualizar o banco `db_ecom`;
- Consultar as tabelas;
- Executar comandos SQL;
- Conferir os registros;
- Criar backups;
- Restaurar backups;
- Realizar manutenção no banco.

Configuração da conexão

- Host: `localhost`
- Porta: `3306`

- Usuário: `root`
- Senha: senha configurada no MySQL
- Banco: `db_ecom`

O MySQL Workbench não é obrigatório para o servidor funcionar. Ele é uma ferramenta gráfica de administração e manutenção. O componente realmente obrigatório é o MySQL Server.

---

8. Visual Studio Code

Versão utilizada

- Visual Studio Code 1.141.0 x64.

O VS Code é recomendado para editar os arquivos do projeto, executar comandos no terminal e testar a API.

Extensões recomendadas

REST Client

- Extensão: `humao.rest-client`
- Versão utilizada: `0.25.1`

Finalidade:

- Executar as requisições do arquivo `backend/rest_client.http`;
- Testar as rotas GET, POST, PUT, PATCH e DELETE;
- Conferir respostas e códigos HTTP;
- Testar autenticação por token.

Live Server

- Extensão: `ritwickdey.LiveServer`
- Versão utilizada: `5.7.10`

Finalidade:

- Executar o frontend por um servidor local;
- Atualizar a página automaticamente;
- Evitar problemas ao abrir os arquivos HTML diretamente.

Essas extensões são importantes para desenvolvimento e manutenção, mas não são necessárias em um servidor de produção.

---

9. Navegador

Navegador testado

- Google Chrome 141.0.7390.108.

Requisito recomendado

- Google Chrome 120 ou superior;
- Microsoft Edge 120 ou superior;
- Mozilla Firefox 120 ou superior.

O navegador precisa oferecer suporte a:

- `fetch`;
- `localStorage`;
- JavaScript moderno;
- Template strings;
- CSS Grid;
- Flexbox;
- Formatação com `toLocaleString`;
- Canvas para os gráficos.

O Google Chrome é o navegador oficialmente utilizado nos testes do projeto.

---

10. Bibliotecas do backend

As bibliotecas são instaladas dentro da pasta `backend` pelo npm.

Biblioteca	Versão	Finalidade
Express	5.2.1	Criação do servidor e das rotas
Sequelize	6.37.8	Comunicação com o banco de dados
mysql2	3.23.4	Driver de conexão com MySQL
bcrypt	6.0.0	Criptografia das senhas
jsonwebtoken	9.0.3	Criação e verificação dos tokens JWT
cors	2.8.6	Liberação das requisições do frontend
dotenv	17.4.2	Leitura das variáveis do arquivo `.env`

Para instalar todas as bibliotecas:

```
cd backend
npm.cmd install
```

Para uma instalação baseada exatamente no `package-lock.json`, pode ser utilizado:

```
npm.cmd ci
```

O projeto não utiliza React, Angular, Vue, TypeScript ou Vite.

---

11. Biblioteca do frontend

O frontend utiliza JavaScript puro e não precisa executar `npm install`.

A única biblioteca externa utilizada é:

- Chart.js 4.4.7.

Ela é carregada pela internet através do endereço CDN configurado em `relatorios.html`.

O Chart.js é utilizado para:

- Gráfico de faturamento mensal;
- Gráfico de estoque por categoria.

Sem conexão com a internet, o restante do sistema pode funcionar localmente, mas os gráficos podem não ser apresentados porque a biblioteca está hospedada externamente.

---

12. Serviços externos

ViaCEP

O sistema utiliza o serviço ViaCEP para consultar o endereço a partir do CEP.

Ele preenche automaticamente:

- Rua;
- Bairro;
- Cidade;
- UF.

Para essa funcionalidade funcionar, o computador precisa possuir acesso à internet.

jsDelivr

O sistema utiliza o CDN jsDelivr para carregar o Chart.js.

Também necessita de conexão com a internet.

---

13. Arquitetura do sistema

O AutoShopping utiliza uma arquitetura cliente-servidor dividida em camadas.

```
Navegador
   ↓
Frontend — HTML, CSS e JavaScript
   ↓ requisições fetch
API REST — Node.js e Express
   ↓
Controllers
   ↓
Models — Sequelize
   ↓
Banco de dados MySQL
```

Frontend

Responsável por:

- Exibir as telas;
- Receber os dados dos formulários;
- Enviar requisições ao backend;
- Exibir veículos, pedidos e relatórios;
- Armazenar carrinho e sessão no `localStorage`.

Backend

Responsável por:

- Receber as requisições;
- Validar os dados;
- Aplicar as regras de negócio;
- Autenticar usuários;
- Controlar permissões;
- Consultar e modificar o banco de dados.

Banco de dados

Responsável por armazenar permanentemente:

- Usuários;
- Categorias;
- Veículos;
- Estoques;

- Pedidos;
- Itens dos pedidos;
- Entregas.

Essa organização pode ser considerada uma arquitetura em camadas semelhante ao padrão MVC:

- **Model:** arquivos da pasta `models`;
- **Controller:** arquivos da pasta `controller`;
- **View:** páginas do frontend;
- **Rotas:** configuradas no `index.js`.

---

14. Estrutura de pastas

```
ecom_autoshopping/
│
├── backend/
│   ├── controller/
│   ├── db/
│   ├── middleware/
│   ├── models/
│   ├── node_modules/
│   ├── .env
│   ├── conn.js
│   ├── sync.js
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   └── rest_client.http
│
└── frontend/
    ├── css/
    ├── html/
    ├── imagens/
    ├── js/
    └── index.html
```

---

15. Variáveis de ambiente

O backend precisa possuir um arquivo chamado `.env`.

Exemplo:

```
DB_NAME=db_ecom
DB_USER=root
DB_PASS=senha_do_mysql
DB_HOST=localhost
DB_PORT=3306

PORT=3000
```

```
JWT_SECRET=uma_chave_secreta_grande_e_segura
JWT_EXPIRES_IN=8h
BCRYPT_SALT_ROUNDS=10
```

Finalidade das variáveis

- `DB_NAME`: nome do banco.
- `DB_USER`: usuário do MySQL.
- `DB_PASS`: senha do MySQL.
- `DB_HOST`: endereço do banco.
- `DB_PORT`: porta do MySQL.
- `PORT`: porta do backend.
- `JWT_SECRET`: chave usada para assinar os tokens.
- `JWT_EXPIRES_IN`: duração do login.
- `BCRYPT_SALT_ROUNDS`: nível de processamento da criptografia.

O arquivo `.env` não deve ser publicado nem enviado para repositórios públicos.

---

16. Portas utilizadas

Serviço	Porta
MySQL	3306
Backend Express	3000
Live Server	Normalmente 5500

Essas portas precisam estar livres.

O frontend realiza requisições para:

```
http://localhost:3000
```

---

17. Configuração inicial

Dentro da pasta do backend:

```
cd backend
npm.cmd install
```

Depois, configure o arquivo `.env`.

Para criar o banco:

```
node conn.js
```

Para sincronizar e criar as tabelas:

```
node sync.js
```

Para iniciar o servidor:

```
node index.js
```

Quando aparecer:

```
Servidor rodando em http://localhost:3000
```

o backend estará funcionando.

Depois, abra o arquivo `frontend/index.html` utilizando o Live Server.

---

18. Sequência para executar o sistema diariamente

Sempre que abrir o projeto:

1. Inicie o serviço do MySQL.
2. Abra o terminal na pasta `backend`.
3. Execute:

```
node conn.js
```

4. Execute:

```
node sync.js
```

5. Execute:

```
node index.js
```

6. Não feche o terminal do `index.js`.
7. Abra o frontend com o Live Server.
8. Acesse pelo Google Chrome.

O `conn.js` e o `sync.js` não precisam permanecer abertos. Somente o `index.js` deve continuar rodando.

---

19. Requisitos de rede

Para funcionamento local:

- MySQL deve aceitar conexão em `localhost:3306`;
- Backend deve estar acessível em `localhost:3000`;
- Frontend deve conseguir enviar requisições para o backend;
- Firewall não deve bloquear Node.js ou MySQL.

Para todas as funcionalidades:

- Acesso à internet para ViaCEP;
- Acesso à internet para Chart.js;
- Acesso à internet para instalar dependências com npm.

---

20. Armazenamento local do navegador

O sistema utiliza `localStorage` para armazenar:

- Token de autenticação;
- Informações básicas do usuário conectado;
- Veículos adicionados ao carrinho.

Chaves utilizadas:

```
autoshopping_token
autoshopping_usuario
autoshopping_carrinho
```

Ao limpar os dados do navegador, o login e o carrinho serão removidos, mas os registros do MySQL continuarão armazenados.

---

21. Requisitos para manutenção

Para realizar manutenção no sistema, recomenda-se possuir conhecimento básico em:

- HTML5;
- CSS3;
- JavaScript;
- Node.js;
- Express;
- API REST;
- Sequelize;
- MySQL;
- JSON;
- Requisições HTTP;

- Git;
- Uso do terminal.

Também é necessário saber:

- Iniciar e parar o servidor;
- Verificar mensagens de erro no terminal;
- Consultar dados no Workbench;
- Testar rotas pelo REST Client;
- Fazer backup do banco;
- Restaurar o arquivo `.env`;
- Reinstalar dependências com npm.

---

22. Backup e segurança

Para manutenção segura, devem ser realizados backups de:

- Banco de dados `db_ecom`;
- Arquivo `.env`;
- Código-fonte;
- Pasta de imagens;
- Arquivo `package.json`;
- Arquivo `package-lock.json`.

Nunca devem ser publicados:

- Senha do banco;
- `JWT_SECRET`;
- Tokens de usuários;
- Dados pessoais reais;
- Arquivo `.env`.

---

23. Resumo do ambiente recomendado

Para executar o AutoShopping corretamente, recomenda-se:

- Windows 10 ou 11 x64;
- Processador com 4 núcleos;
- 8 GB de RAM;
- 5 GB de espaço livre;
- Node.js 22.x;
- npm 10.x;
- MySQL Server 8.0;
- MySQL Workbench 8.0;

- Google Chrome 120 ou superior;
- Visual Studio Code;
- REST Client;
- Live Server;
- Conexão com a internet;
- Portas 3000 e 3306 disponíveis;
- Arquivo `.env` corretamente configurado.